VOLCANO-CA-MC: DOKUMENTACJA TECHNICZNA

Instalacja, budowanie i uruchamianie

Bartosz Pajda

Wygenerowano: 27.10.2026

Krok po kroku na świeżym komputerze z Windows (GCC/Linux: punkt 2.4). Ten plik wystarcza, żeby od zera zbudować silnik, uruchomić testy, symulacje z linii poleceń, GUI, serię „CA vs MC", pomiary profilu i przejść krótkie demo. Polecenia są w składni **PowerShell** (domyślny terminal VS Code i Windows Terminal) i działają też w `cmd`, chyba że zaznaczono inaczej. Wszystkie wykonuj **z katalogu projektu `volcano-ca-mc`** (tam, gdzie leży `CMakeLists.txt`). Jak to działa w środku: **dokument „Jak to działa: architektura i kluczowe miejsca w kodzie"**. Opis wszystkich opcji silnika, parametrów configu i plików wynikowych: punkt 10.

1. Wymagane oprogramowanie

Program	Skąd	Co zaznaczyć przy instalacji	Sprawdzenie
Visual Studio 2022 Community	visualstudio.microsoft.com → „Pobierz Visual Studio"	pakiet roboczy **„Programowanie aplikacji klasycznych w języku C++"**; w szczegółach zostaw zaznaczone **„Narzędzia CMake języka C++ dla systemu Windows"** i **„Windows 11 SDK"**	w menu Start jest „Visual Studio 2022"
CMake ≥ 3.20 (osobno, polecane)	cmake.org/download → „Windows x64 Installer"	**„Add CMake to the system PATH for all users"**	nowy terminal: `cmake --version`
Git for Windows	git-scm.com	domyślne; daje też Git Bash	`git --version`
Python 3.12	python.org/downloads	**„Add python.exe to PATH"**; „Install for all users" (wtedy pakiety instaluj z `--user`)	`python --version`
VS Code (opcjonalnie)	code.visualstudio.com	rozszerzenia: **C/C++**, **CMake Tools**, **Python**	—
WSL + Ubuntu (opcjonalnie, GCC i valgrind)	PowerShell jako administrator: `wsl --install -d Ubuntu-22.04`, restart	w Ubuntu: `sudo apt update && sudo apt install -y build-essential cmake valgrind`	`wsl -l -v` (Ubuntu, wersja 2); w Ubuntu `g++ --version`
ParaView (opcjonalnie, pliki VTK)	paraview.org/download	domyślne	otwiera pliki `.vti`
MS-MPI 10.1 (opcjonalnie, wersja MPI)	strona Microsoft „Microsoft MPI v10.1.3", **oba pliki**: `msmpisetup.exe` (`mpiexec`) i `msmpisdk.msi` (nagłówki, `msmpi.lib`)	domyślne; potem zamknij i otwórz terminal / VS Code (nowe zmienne `MSMPI_*`)	`mpiexec -help`; `echo %MSMPI_INC%` pokazuje ścieżkę
OpenMPI w WSL (opcjonalnie, wersja MPI)	w Ubuntu: `sudo apt install -y openmpi-bin libopenmpi-dev`	brak	w Ubuntu `mpicxx --version`, `mpirun --version`

Uwagi: - Jeśli CMake instalowało tylko Visual Studio (bez osobnego instalatora), `cmake` nie jest w `PATH`; wtedy używaj pełnej ścieżki (punkt 2.3) albo dopisz `C:\Program Files\CMake\bin` do zmiennej `PATH`. - Po instalacji zamknij i otwórz terminal (nowy `PATH`).

Sprawdzenie w nowym terminalu (wersje, z którymi projekt był sprawdzany: CMake 3.29.0, Python 3.12.1, Git 2.43.0, Ubuntu-22.04 w WSL 2):

```
cmake --version
python --version
git --version
wsl -l -v
```

2. Budowanie silnika

Program silnika po zbudowaniu (MSVC): **`build\Release\engine.exe`** (Debug: `build\Debug\engine.exe`), testy `build\Release\engine_tests.exe`. Presety budowania są w `CMakePresets.json`; **pomiary czasu zawsze w Release**.

2.1 VS Code (CMake Tools)

1. *File* → *Open Folder* → folder `volcano-ca-mc`.
2. CMake Tools wykryje `CMakePresets.json` i zapyta o **Configure Preset** → wybierz **`msvc-release`** (MSVC Release x64). Jeśli nie zapyta: paleta poleceń (Ctrl+Shift+P) → „CMake: Select Configure Preset".

3. Na dolnym pasku **Build** (albo F7). W panelu *Output* na końcu: `engine.vcxproj -> …\build\Release\engine.exe`.
4. Testy: paleta → „CMake: Run Tests” (albo `ctest` z punktu 3).

(Gdy presety są wyłączone: kit „Visual Studio Community 2022 Release – amd64”, wariant **Release**.)

2.2 Visual Studio 2022

1. *Plik → Otwórz → Folder...* → `volcano-ca-mc` (projekt CMake wykrywa się sam, chwilę trwa „konfigurowanie”).
2. Na pasku narzędzi lista konfiguracji → **`msvc-release`** (jeśli widać „x64-Debug”/„x64-Release”, to stara konfiguracja `CMakeSettings.json`; wybierz preset z `CMakePresets.json`, patrz FAQ).
3. *Kompiluj → Kompiluj wszystko* (Ctrl+Shift+B). Wynik: `build\Release\engine.exe`.
4. Testy: *Test → Uruchom testy CTest*; profiler: *Debuguj → Profiler wydajności* (Alt+F2), szczegóły w `docs/profilowanie.md`.

2.3 Linia poleceń (bez środowiska vcvars)

```
cmake --preset msvc-release
cmake --build --preset msvc-release
ctest --preset msvc-release
```

Jeśli `cmake` nie jest w `PATH` (PowerShell: operator `&` przed ścieżką w cudzysłowie):

```
& "C:\Program Files\CMake\bin\cmake.exe" --preset msvc-release
& "C:\Program Files\CMake\bin\cmake.exe" --build --preset msvc-release
& "C:\Program Files\CMake\bin\ctest.exe" --preset msvc-release
```

(w `cmd` bez `&`: `"C:\Program Files\CMake\bin\cmake.exe" --preset msvc-release`). Build Debug: `cmake --build --preset msvc-debug`, testy `ctest --preset msvc-debug` (Debug jest kilka razy wolniejszy; służy do wyłapywania błędów: asercje sprawdzają dostęp poza ramką). Pierwsze budowanie trwa 1–2 min.

OpenMP jest włączone domyślnie (opcja CMake `VOLCANO_OPENMP=ON`; MSVC: `/openmp`, OpenMP 2.0; GCC: libgomp). Build bez OpenMP w osobnym katalogu:

```
cmake -S . -B build-noomp -G "Visual Studio 17 2022" -A x64 -DVOLCANO_OPENMP=OFF
cmake --build build-noomp --config Release --parallel 4
```

(GCC: `cmake -S . -B build-gcc-noomp -DCMAKE_BUILD_TYPE=Release -DVOLCANO_OPENMP=OFF && cmake --build build-gcc-noomp -j 4`). To ten sam program w jednym wątku: `--threads` wypisuje wtedy ostrzeżenie „program zbudowano bez OpenMP”. Cel `omp_overhead` (mikrotest narzutu OpenMP, punkt 7.2) buduje się razem z silnikiem, gdy OpenMP jest włączone.

MPI: program `engine_mpi` (wersja rozproszona, punkt 4.2) i jego testy budują się razem z silnikiem, gdy CMake znajdzie MPI (opcja `VOLCANO_MPI=ON`, domyślnie): na Windows potrzebne jest **SDK** MS-MPI (konfiguracja wypisuje `Found MPI_CXX: …/msmpi.lib`), w WSL `libopenmpi-dev`. Bez MPI konfiguracja wypisuje komunikat i buduje resztę bez zmian; build z wyłączonym MPI: `-DVOLCANO_MPI=OFF` (program `engine` jest ten sam).

2.4 GCC w WSL

Z **PowerShell**, `cmd` albo Git Bash, **z katalogu projektu** (`wsl` startuje w bieżącym katalogu Windows, np. `/mnt/c/…/volcano-ca-mc`, więc ścieżka nie jest potrzebna; bez zagnieżdżonych cudzysłowów polecenie działa też w Windows PowerShell 5.1):

```
wsl -e bash -lc "cmake --preset gcc-release && cmake --build --preset gcc-release -j 16 &&
ctest --preset gcc-release"
```

albo w oknie Ubuntu: `cd` do katalogu projektu (`/mnt/<dysk>/<ścieżka>/volcano-ca-mc`, ścieżkę ze spacjami w cudzysłowie) i te same trzy polecenia. Na Linuksie bez WSL te same trzy polecenia w katalogu projektu. Program: `build-gcc-release/bin/engine`; Debug: preset `gcc-debug`.

3. Testy

```
ctest --preset msvc-release            # MSVC Release (≈ 30 s; z testami MPI ≈ 2,5 min)
ctest --preset msvc-debug              # MSVC Debug (≈ 3,5 min; z testami MPI ≈ 13 min)
python -m pip install --user -r gui\requirements-dev.txt   # raz: pytest, httpx, playwright + pakiety GUI
python -m pytest gui\tests             # backend GUI z prawdziwym silnikiem (≈ 12 s)
```

GCC: `ctest --preset gcc-release` / `gcc-debug` (w WSL, jak w 2.4). Poprawny wynik:

```
100% tests passed, 0 tests failed out of 21
...
102 passed in 11.44s
```

`ctest` to 12 testów: `unit_tests` (123 przypadki doctest, ~318 tys. asercji: sąsiedztwa, brzegi, zgodność vector/macierz, reguły lawy, MC, config, zapis/odczyt, `--batch`, kratery z listy `vents.positions`, odtwarzanie erupcji = stan automatu w każdym kroku, migawki CA i MC, automat frontalny = pełny przebieg, wyniki CA i szachownicy MC bitowo te same dla 1, 2, 3, 4 i 8 wątków i każdego harmonogramu, kolory szachownicy, podział na pasy i plan warstw brzegowych) i 11 testów dymnych programu (pomoc, symulacja 2D, błędny config, MC, `--mc-from`, `--print-defaults`, `--validate`, `--progress`, `--batch`), a gdy jest MPI: 9 testów wersji rozproszonej: `mpi_np1`, `np2`, `np3`, `np4`, `np5`, `np8` (testy `engine_mpi_tests` pod `mpiexec -n N`: wynik bitowo ten sam co `engine`, także pliki wyników każdej kombinacji `mpi.pack` × `mpi.bcast` × `mpi.io` bajt po bajcie; MSVC Release ≈ 20–25 s każdy, Debug ≈ 80–100 s, GCC ≈ 9–13 s) i 3 dymne `mpi_cli_*` (lawa + MC na 3 procesach, `--mc-from`, odrzucenie `mc.update = random` przy 2 procesach). Bez MPI wynik to `out of 12`. `pytest` obejmuje też `tools\fetch_dem.py` (bez internetu, sztuczne kafelki) i odtwarzanie erupcji (migawki silnika, endpointy, ta sama reguła w JS przez `node`, jeśli jest zainstalowany). Jeśli `pytest` pisze „skipped”, silnik nie jest zbudowany (testy z silnikiem są wtedy pomijane).

4. Silnik z linii poleceń

```
.\build\Release\engine.exe --help
.\build\Release\engine.exe --config configs\test_2d.json --out results\test_2d
.\build\Release\engine.exe --config configs\test_3d.json --out results\test_3d_macierz --grid matrix
.\build\Release\engine.exe --config configs\classic_ref.json --out results\classic_moore --set space.nx=50 --set neighborhood=moore
.\build\Release\engine.exe --config configs\lava_mc_2d.json --out results\lava_mc_2d
.\build\Release\engine.exe --mc-from results\lava_mc_2d --config configs\mc_basalt.json --out results\mc_from_lava_2d
.\build\Release\engine.exe --config configs\heightmap_kaldera.json --out results\heightmap_kaldera
.\build\Release\engine.exe --validate --config configs\test_3d.json
.\build\Release\engine.exe --validate --set space.nx=-5
.\build\Release\engine.exe --print-defaults
.\build\Release\engine.exe --benchmark --quick
.\build\Release\engine.exe --config configs\test_3d.json --out results\test_3d_8w --threads 8 --set parallel.affinity=pcores
.\build\Release\engine.exe --config configs\test_3d.json --out results\test_3d_full_8w --threads 8 --set ca.sweep=full --set parallel.schedule=guided --thread-stats
.\build\Release\engine.exe --config configs\mc_full_2d.json --out results\mc_szachownica --threads 4 --set mc.update=checkerboard
```

Polecenie	Co robi
`--config <plik> --out <folder>`	symulacja z configu; folder wyników tworzony automatycznie (bez `--out`: `results\<nazwa configu>`)
`--set klucz=wartość` (wiele razy)	zmiana parametru bez edycji pliku (`space.nx=50`, `neighborhood=moore`, `mc.mcs=20` …)
`--grid matrix`	reprezentacja siatki `vector<vector<vector>>` zamiast płaskiej tablicy (wynik bitowo ten sam)
`--mc-from <folder>`	samo Monte Carlo na gotowym wyniku CA (`ca_result.bin` / `.txt` z folderu); parametry MC z configu
`--batch <folder> --out <folder>`	seria: wszystkie configi `*.json` z folderu, każdy w osobnym procesie, `batch_summary.csv` (punkt 6.2)

Polecenie	Co robi
`--validate`	tylko sprawdzenie configu: JSON `{"ok", "errors", "warnings"}`, kod 0 albo 1
`--print-defaults`	pełny domyślny config (wszystkie dozwolone klucze)
`--benchmark [--quick] [--repeats N]`	vector vs macierz: czasy i zgodność wyników (`--quick` bez przypadków ≥ 1 mln komórek, ~10 s; wyniki w `results\benchmark\`)
`--timing-detail`, `--progress`	czasy faz wewnątrz CA/MC w `times.json`; linie postępu (dla GUI)
`--threads N`	liczba wątków OpenMP (skrót `--set parallel.threads=N`; domyślnie 1, kod sekwencyjny; 0 = wszystkie); wynik CA bitowo ten sam dla każdej liczby wątków
`--set parallel.affinity=pcores`	jeden wątek na rdzeń fizyczny wydajny (P), do pomiarów; `parallel.schedule`, `parallel.chunk`: harmonogram pętli
`--thread-stats`	czas pracy każdego wątku w fazach kroku i nierównomierność obciążenia → `times.json`, sekcja `threadStats`
`--set mc.update=checkerboard`	MC w trybie szachownicy (liczy się w wątkach; wynik ten sam dla każdej liczby wątków, statystycznie równoważny wariantowi naiwnemu `random`)

Kody wyjścia: 0 OK, 1 błąd configu, 2 błąd argumentów, 3 inny błąd, 4 niezgodne wyniki benchmarku, 5 seria z błędem. Czasy (MSVC Release, wolny komputer): `test_2d` ~0,15 s, `test_3d` ~2 s, `heightmap_kaldera` ~3 s, `etna_3d` ~15 s, `etna_real` / `etna_real_boczna` ~20 s, `wezuwiusz_real` ~5 s (punkt 4.1).

Pliki w folderze wyników:

Plik	Zawartość
`config_used.json`	pełny config po scaleniu z wartościami domyślnymi i `--set`
`ca_result.txt` / `ca_result.bin`	mikrostruktura CA: `x y z type flowId temp` (tekst) / binarnie wszystkie pola (13 B/komórkę); format wg `output.format` (`txt`, `bin`, `both`, `none`)
`mc_result.txt` / `mc_result.bin`	kryształy MC (`x y z crystal`, 0 poza domeną)
`mc_stats.csv`	liczba ziaren, średnia wielkość, energia, akceptacja co `mc.statsEvery` MCS
`times.json`	czasy faz [ms] (`readConfig, allocate, terrain, vents, caSim, save, mcInit, mcSim, mcStats, mcSave, total`; z migawkami także `snapshots`, `mcSnapshots`), kroki, procesor, kompilator, pamięć (`memory`), rozmiary plików (`files`)
`snapshots\step_<k>.bin`	(tylko `output.snapshotsEvery` > 0) stan CA po k krokach, format `ca_result.bin`; k = 0, co N kroków i ostatni
`mc_snapshots\mcs_<k>.bin`	(tylko `mc.snapshotsEvery` > 0) kryształy po k MCS, format `mc_result.bin`; MCS 0, co N i ostatni; odtwarzanie wzrostu ziaren w GUI
`stats.json`	objętość lawy na krater, zasięg, historia kroków; sekcja `mc` (ziarna, energia, efekt mobilności)
`engine.log`	(zadania z GUI i `--batch`) całe wyjście programu

Typy komórek: 0 powietrze, 1 skała, 2 lawa (w `classic`: ziarno), 3 bazalt.

Podgląd bez GUI (Python z `numpy` i `matplotlib`: `python -m pip install --user numpy matplotlib`):

```
python tools\plot_slice.py results\test_2d
python tools\plot_slice.py results\lava_mc_3d --axis top --color crystal
python tools\plot_mc_stats.py results\lava_mc_2d
python tools\export_vtk.py results\lava_mc_3d
```

`plot_slice.py` zapisuje `<folder>\slice.png` (opcje: `--axis x|y|z|top`, `--index`, `--color type|flow|temp|cooling|crystal`, `--boundaries`, `--out`), `plot_mc_stats.py`: `mc_stats.png` (liczba ziaren i średnie ziarno vs MCS), `export_vtk.py`: `<folder>\<nazwa>.vti` do ParaView (*File → Open → Apply*, kolor wg `crystal` / `flowId`, filtr *Threshold* `type ≥ 2`). `results\lava_mc_3d` powstaje z `.\build\Release\engine.exe --config configs\lava_mc_3d.json --out results\lava_mc_3d`.

4.1 Prawdziwe wulkany: Etna i Wezuwiusz

```
.\build\Release\engine.exe --config configs\etna_real.json --out results\etna_real
# ~20 s, 3 kratery szczytowe
```

```
.\build\Release\engine.exe --config configs\etna_real_boczna.json --out
results\etna_real_boczna    # ~20 s, szczelina na S zboczu
.\build\Release\engine.exe --config configs\wezuwiusz_real.json --out results\wezuwiusz_real
# ~5 s
python tools\plot_slice.py results\etna_real --axis top --color flow
# widok z góry
```

Mapy terenu są już w repozytorium (`configs\heightmaps\etna.*`, `wezuwiusz.*`: CSV, podgląd PNG, metadane JSON; przypisanie danych w `configs\heightmaps\README.md`). Pobranie od nowa / inny obszar (internet, kilka MB kafelków do `tools\.dem_cache\`, poza repozytorium):

```
python tools\fetch_dem.py --name etna --bbox 37.58 14.80 37.92 15.22 --zoom 11 --nx 160 --ny
160 --nz 70
python tools\fetch_dem.py --name wezuwiusz --bbox 40.77 14.36 40.87 14.49 --zoom 12 --nx 140
--ny 140 --nz 50
python tools\fetch_dem.py --name etna --bbox 37.58 14.80 37.92 15.22 --zoom 11 --nx 160 --ny
160 --nz 70 --from-cache   # bez internetu
python tools\latlon_to_grid.py etna 37.751 14.993 37.747 15.000 --window 3   # (szer., dł.) -
> kolumna (x, y), okno wysokości
python tools\latlon_to_grid.py etna --cell 74 82                              # kolumna ->
(szer., dł.)
```

Kratery w zadanych miejscach w dowolnym configu: `"vents": {"placement": "positions", "positions": [[73, 80], [74, 82]], "count": 2}` (`count` = długość listy) albo z linii poleceń: `--set vents.placement=positions --set "vents.positions=[[30,0]]" --set vents.count=1` (przekrój 2D: y = 0).

Migawki do sprawdzenia odtwarzania (domyślnie wyłączone, nie wliczają się do `caSim` / `mcSim`):

```
.\build\Release\engine.exe --config configs\lava_mc_3d.json --out results\replay_lava_mc_3d -
-set output.snapshotsEvery=20 --set mc.snapshotsEvery=10
```

Opis metody, wyników i ograniczeń: **dokument „Prawdziwy teren i odtwarzanie erupcji”**.

4.2 Wersja rozproszona: `engine_mpi` (MPI)

Ta sama symulacja i te same pliki wyników co `engine`, ale przestrzeń jest podzielona na pasy warstw między procesy MPI (wynik bitowo ten sam przy każdej liczbie procesów). MC przy więcej niż 1 procesie tylko szachownicą (`mc.update = checkerboard`, bo wariant naiwny jest sekwencyjny; program odmawia z komunikatem).

```
mpiexec -n 4 .\build\Release\engine_mpi.exe --config configs\test_3d.json --out
results\test_3d_mpi4
mpiexec -n 3 .\build\Release\engine_mpi.exe --config configs\lava_mc_2d.json --out
results\lava_mc_2d_mpi3 --set mc.update=checkerboard
mpiexec -n 8 .\build\Release\engine_mpi.exe --config configs\test_3d.json --out
results\test_3d_mpi8 --high-priority --set parallel.affinity=pcores
mpiexec -n 2 .\build\Release\engine_mpi.exe --config configs\test_3d.json --out
results\test_3d_mpi2x4 --threads 4 --high-priority --set parallel.affinity=pcores
mpiexec -n 3 .\build\Release\engine_mpi.exe --config configs\test_3d.json --out
results\test_3d_mpi3c --set mpi.exchange=changed
wsl -e mpiexec --allow-run-as-root --oversubscribe --bind-to none -n 3 build-gcc-
release/bin/engine_mpi --config configs/test_3d.json --out results/test_3d_mpi3_gcc
```

- Na końcu proces 0 wypisuje podsumowanie z liczbą komunikatów i bajtów wymiany; `times.json` ma sekcję `mpi` (podział warstw, czasy komunikacji i obliczeń każdego procesu, bajty, procesory i priorytet procesów).
- **Do pomiarów** `--high-priority` (`mpiexec` uruchamia procesy z priorytetem normalnym) i `parallel.affinity = pcores` przypina sam program: proces r z T wątkami na rdzeniach P nr r·T … r·T + T − 1 (procesory logiczne 2, 4, …, 14, 0). `mpiexec -affinity` MS-MPI nie rozróżnia rdzeni P i E, nie używać. Łącznie najwyżej 8 procesów × wątków.
- WSL: OpenMPI w maszynie wirtualnej pracuje jako `root` (`--allow-run-as-root`) i widzi 4 rdzenie (8 procesów: `--oversubscribe`); `--bind-to none`, bo przypina program. Ścieżki w WSL względem katalogu projektu (z Git Bash przed `wsl` ustaw `MSYS_NO_PATHCONV=1`).
- Opis: **dokument „Wersja rozproszona MPI”**.

Typy pochodne MPI i MPI-IO: trzy opcje, każda osobno albo razem (domyślnie jak w wersji MPI bez nich; pliki wyników bajtowo te same):

```
mpiexec -n 4 .\build\Release\engine_mpi.exe --config configs\test_3d.json --out
results\test_3d_mpi4_dsi --set mpi.pack=datatype --set mpi.bcast=struct --set mpi.io=mpiio
mpiexec -n 3 .\build\Release\engine_mpi.exe --mc-from results\lava_mc_2d_mpi3 --config
configs\mc_basalt.json --out results\mc_from_mpi3_io --set mc.update=checkerboard --set
mpi.io=mpiio
wsl -e mpiexec --allow-run-as-root --oversubscribe --bind-to none -n 3 build-gcc-
release/bin/engine_mpi --config configs/test_3d.json --out /tmp/test_3d_mpi3_io --set
mpi.io=mpiio
wsl -e cp -r /tmp/test_3d_mpi3_io results/
```

- `mpi.pack=datatype`: wymiana warstw brzegowych typem pochodnym (bez bufora); `mpi.bcast=struct`: parametry i kratery jako struktury; `mpi.io=mpiio`: każdy proces zapisuje swój pas do wspólnego pliku, a przy `--mc-from` (plik `ca_result.bin`) czyta swój pas. Podsumowanie procesu 0 i `times.json` (`mpi.pack`, `mpi.bcast`, `mpi.io`) mówią, który wariant działał.
- **WSL: MPI-IO tylko na ext4** (`/tmp` albo katalog w systemie plików WSL, potem kopia, jak wyżej); na dysku Windows (`/mnt/<dysk>`, katalog projektu) MPI-IO działa, ale wielokrotnie wolniej. MS-MPI pisze na NTFS bez ograniczeń.
- Opis: **dokument „Typy pochodne MPI i równoległy zapis”**.

5. GUI

5.1 Instalacja i start

```
python -m pip install --user -r gui\requirements.txt    # raz: fastapi, uvicorn, numpy,
pillow
python -m gui                                            # http://127.0.0.1:8000 + otwarcie
przeglądarki
```

albo dwuklik **`gui\run_gui.bat`** (sam doinstaluje brakujące pakiety). Zatrzymanie: **Ctrl+C** w oknie konsoli. Opcje: `python -m gui --port 8001 --no-browser`. GUI działa bez internetu (three.js, Chart.js i fonty są w `gui\static\vendor\`) i tylko na tym komputerze (127.0.0.1). Silnik jest wykrywany sam (`build\Release\engine.exe`, potem inne buildy); inna ścieżka, port, liczba workerów, procesory do przypinania: `gui\settings.json` (wzór `gui\settings.example.json`).

5.2 Zakładki: co kliknąć

Nagłówek: zakładki w tej samej kolejności, wizualnie pogrupowane w kroki pracy: *Przygotuj* (Nowa symulacja, Seria), *Licz* (Zadania, z liczbą aktywnych zadań), *Analizuj* (Wyniki, Porównanie, Podgląd 2D, Podgląd 3D, Czasy i statystyki, Wydajność); przy węższym oknie etykiety grup się chowają, zostają cienkie separatory. Po prawej status silnika (zielona kropka „silnik gotowy”, pełna ścieżka w podpowiedzi) i **przełącznik motywu** (księżyc / słońce): ciemny „bazalt” albo jasny „popiół” (do jasnej sali i zrzutów do druku). Wybór zapamiętuje przeglądarka; bez wyboru motyw idzie za ustawieniem systemu. Eksport PNG / klatek / WebM ma kolory bieżącego motywu. Opis: `docs\wyglad-interfejsu.md`.

1. **Nowa symulacja**: *Preset* → np. `test_3d` (wypełnia formularz) → zmień parametry (pomarańczowa kropka = inna niż domyślna, szare pola nie dotyczą trybu) → **Sprawdź** (silnik waliduje; błąd po polsku pod polem i nad przyciskami) → **Uruchom** (powstaje `configs\gui\<id>.json` i `results\gui\<id>\`; ścieżki pokazane pod przyciskiem). Panel po prawej: szacunek pamięci i czasu. *Importuj z pliku…* / *Eksportuj do JSON*: config z/do pliku. *Źródło danych*: samo MC na gotowym wyniku CA (`--mc-from`). **Mapa wysokości terenu:** lista map z `configs\heightmaps\` (miniatura: jasne = wyżej); wybór ustawia teren z pliku i wymiary siatki; **Importuj mapę (CSV / PNG / JPG)…** → wybierz plik → (obraz: nx, ny, nz, wysokość maks., podstawa, wygładzanie, odwróć) → **Importuj do configs/heightmaps/**: mapa trafia na listę i do formularza. Przykład obrazu: `configs\heightmaps\kaldera_wylom.png`. Mapy z prawdziwego terenu (`etna`, `wezuwiusz`) pokazują obszar, skalę i źródło. **Kratery na mapie**: *Preset* `etna_real` / `wezuwiusz_real` albo *Rozmieszczenie kraterów* = „w zadanych miejscach” → panel z mapą (kolory wysokości, cieniowanie, poziomice): **kliknięcie dodaje krater, kliknięcie na**

znaczniku usuwa (*Cofnij ostatni*, *Wyczyść*); pod kursorem kolumna, wysokość w m n.p.m. i szer./dł. geograficzna; lista trafia do `vents.positions`, liczba do `vents.count`.

2. **Seria**: *Gotowe szablony* (np. „CA vs MC: rozmiar siatki") albo własna: *Config bazowy* (formularz / preset / plik) → *+ dodaj parametr* → klucz z listy (kolejny klucz w tej samej osi = **sparowanie**) → wartości (lista `100, 200`, krotki `100;100;1`, etykiety `[CA] false;basalt`, zakres, potęgi) → tabela podglądu, łączny czas i pamięć → **Uruchom serię** (najpierw wszystkie pliki, potem kolejka). Przy szablonach pomiarowych: **Ustaw 1 worker + przypięcie**.
3. **Zadania**: kolejka: *Wstrzymaj/Wznów kolejkę*, *Workery* (1 = po kolei; więcej: ostrzeżenie o pomiarach), *przypnij silnik do rdzenia P*, filtr statusu, *grupuj po serii* (postęp „X z N", czas do końca, *Anuluj serię*, *Porównanie*). Przy zadaniu: ▲ ▼ (kolejność oczekujących), *Log*, *Anuluj*, *Ponów* (po błędzie), *Pokaż wynik*.
4. **Wyniki**: wszystkie wyniki (`results\…`, GUI, serie), filtr, sortowanie; *2D*, *3D*, *Czasy*; **ZIP** (cały folder), **VTK** (plik `.vti`), **MC** (samo MC na tym wyniku lawy).
5. **Porównanie**: *Seria* (z GUI albo z linii poleceń) albo „porównaj dowolne wyniki z listy" → wykres (X, Y, serie, osobne wykresy, log X/Y, *Widok domyślny*, **Eksport PNG**), tabela (zestawy kolumn, sortowanie, **Eksport CSV**), galeria miniatur (klik → podgląd 2D, „widok 3D").
6. **Podgląd 2D**: wybór wyniku na pasku nad podglądem, *Pole* (typ, krater/ziarno, temperatura, kryształ, czas stygnięcia), przekrój ⊥ x/y/z, suwak warstwy, granice ziaren, kółko: zoom, przeciąganie: przesuwanie, podpowiedź z wartościami komórki, *Eksport PNG*. Na pasku: *Pobierz ZIP*, *Eksport VTK*. **Odtwarzanie erupcji** (wyniki z `ca_result.bin`): włącz przełącznik *odtwarzanie erupcji* → pasek z ▶ **Odtwórz / ⏸ Pauza** (także **spacja**), suwakiem „krok 0 … ostatni", prędkością (kroków/s), numerem kroku i liczbą komórek płynnych/zastygłych (całość i przekrój); *kolor*: żar (temperatura w tym kroku) albo krater; suwak warstwy działa dalej; podpowiedź pokazuje stan komórki w kroku. **Klatki PNG (ZIP)**: do 200 klatek, **Nagraj WebM**: film 25 kl./s. Gdy wynik ma migawki MC (`mc.snapshotsEvery`), źródło „krystalizacja (MC)" odtwarza wzrost ziaren.
7. **Podgląd 3D**: *Widok*: *ściany* (6 ścian bloku; górna jako rzut z góry), *powierzchnia* (zbocze z lawą, przewyższenie), *woksele*; lewy przycisk: obrót, prawy: przesuwanie, kółko: zoom; *Reset kamery*, *Eksport PNG* (po prawej nad podglądem). Tło: nocne niebo (motyw ciemny) albo mglisty dzień (jasny); płynna lawa w odtwarzaniu świeci. **Odtwarzanie erupcji** jak w 2D w każdym z trzech widoków (powierzchnia = najwyższa niepusta komórka kolumny w danym kroku); dane pobierane raz, klatki liczy przeglądarka, więc można obracać kamerę w trakcie odtwarzania.
8. **Czasy i statystyki**: `times.json` (czas każdej fazy, % całości), pamięć (pomiar i model), pliki, statystyki CA i MC, wykresy liczby ziaren i średniego ziarna vs MCS.
9. **Wydajność**: wyniki optymalizacji i zrównoleglenia bez otwierania plików: nagłówek z zasadą zgodności bitowej i panelem **Pojęcia** (słowniczek), zakładki **Podsumowanie**, **Optymalizacja sekwencyjna**, **OpenMP**, **MPI**, **Typy MPI i MPI-IO**, **Testy wydajności**; po prawej **Kompilator: MSVC | GCC**. Przy wykresie: przełączniki (przypadek, miara…), podpowiedź z dokładną wartością po najechaniu, **Tabela**, **Eksport PNG**, wniosek, **Jak czytać**, warunki pomiaru, **Skąd dane** (zwinięte) i **Pobierz PDF** dokumentu. Dane: `gui\static\data\wydajnosc.json`; po nowych pomiarach odśwież je `python tools\wydajnosc_gui.py` (punkt 7.6; PDF dokumentów: punkt 7.7).

Stan zadań i kolejki: `results\gui\jobs.json` (przeżywa restart). Jedno GUI na katalog projektu.

6. Seria i porównanie CA vs MC

6.1 W GUI

1. `python -m gui` → zakładka **Seria** → szablon **„CA vs MC: rozmiar siatki"** (16 zadań).
2. **Ustaw 1 worker + przypięcie** (żółta ramka po prawej): równoległe zadania i rdzenie E zaburzają czasy.
3. **Uruchom serię** → komunikat z folderami `configs\gui\series\<id>\` i `results\gui\series\<id>\`.

4. **Zadania**: grupa serii: postęp, czas do końca (cała seria ≈ 20–25 s na wolnym komputerze). Nie obciążaj w tym czasie komputera: zamknij gry, programy działające w tle (np. narzędzia producenta laptopa, które przez WMI potrafią stale zajmować ~2 procesory), przeglądarkę z wideo: przy grze w tle czasy MC były do 2,8× dłuższe.
5. **Porównanie** → seria jest wybrana → wykres „czas symulacji vs liczba komórek", log–log, 2D i 3D obok siebie → **Eksport PNG**, **Eksport CSV**; zmień *Y* na „koszt jednostkowy": ns na komórkę i krok (CA) i ns na próbę (MC).

Szablon **„OpenMP: liczba wątków"** liczy lawę `test_3d` (pełny przebieg) i MC szachownicę przy 1, 2, 4, 6 i 8 wątkach; wykres porównania: czas vs wątki. Zadanie wielowątkowe GUI uruchamia z wysokim priorytetem i **nie** przypina go do jednego procesora nawet przy włączonym przypinaniu (wątki rozkłada silnik na rdzenie P, `parallel.affinity = pcores` w szablonie); dalej 1 worker. Log zadania: „wątki OpenMP: N, priorytet wysoki".

To samo automatycznie (Chromium sterowany skryptem, te same kliknięcia; wynik do `docs\img\ca_vs_mc.png`, `docs\data\ca_vs_mc.csv` i `docs\data\ca_vs_mc_meta.json` z obciążeniem tła):

```
python -m playwright install chromium     # raz
python gui\tests\run_ca_vs_mc.py
```

6.2 Bez GUI: `tools\make_series.py` + `engine --batch`

```
python tools\make_series.py --list-templates
python tools\make_series.py --template ca_vs_mc --dry-run
python tools\make_series.py --template seeds --run --cpu 2
python tools\make_series.py --template ca_vs_mc --name ca_vs_mc_mala --run --cpu 2 --vary
"rozmiar:space.nx,space.ny,space.nz=[2D_100x100] 100;100;1, [2D_200x200] 200;200;1,
[3D_10x10x10] 10, [3D_25x25x25] 25"
```

`make_series.py` zapisuje configi w `configs\series\<id>\` (+ `series.json`) i foldery w `results\series\<id>\`, z `--run` uruchamia `engine --batch` i na końcu wypisuje tabelę (zapisuje też `porownanie.csv`). Bez `--run` wypisuje polecenie do ręcznego uruchomienia, np.:

```
.\build\Release\engine.exe --batch configs\series\20261010-025742_ca_vs_mc_mala --out
results\ca_vs_mc_ponownie
```

(identyfikator serii, data-czas i nazwa, jest w komunikacie `make_series.py`). `engine --batch` liczy po kolei wszystkie `*.json` z folderu (pomija `series.json` i `_*.json`), każdy w osobnym procesie, wyniki do `<out>\<nazwa configu>\`, a `<out>\batch_summary.csv` zawiera status, kroki, czasy, pamięć i rozmiary plików. Własne parametry: `--base configs\classic_ref.json --set space.nz=1 --vary "space.nx,space.ny=100, 200" --pow "vents.count=1:100:10"`. Seria z linii poleceń jest też widoczna w zakładce **Porównanie** GUI.

7. Pomiary i profil

```
python bench\run_profile.py --quick --out results\profil_test --name profil_quick   # szybkie
sprawdzenie (~0,5 min)
python bench\plot_profile.py --csv results\profil_test\profil_quick.csv --out
results\profil_test\img
python bench\run_profile.py --full --gcc                                  # pełna macierz MSVC +
GCC/WSL (~25 min)
python bench\plot_profile.py                                              # wykresy
docs\img\profile_*.png z zestawu odniesienia
```

- `run_profile.py` generuje configi, uruchamia `engine` (MSVC Release; `--gcc`: drugi przebieg w WSL) z przypięciem do procesora logicznego 2 (rdzeń P, `--cpu`), każdy wariant 3 razy, i zapisuje `<nazwa>.csv`, `_raw.csv`, `_meta.json` (sprzęt, obciążenie tła), `_benchmark.csv`; domyślnie w `bench\results\`, nazwa domyślnie `profile_<data>[_quick]` (dlatego w przykładzie stała nazwa `--name profil_quick`).

- Sprawdzone 10.10.2026: pierwsze dwa polecenia i ostatnie (`plot_profile.py` bez opcji odtwarza `docs\img\profile_*.png` co do piksela). **Nie uruchamiane ponownie:** `--full --gcc` (~25 min) i profilery z punktu niżej: to pomiary profilu aplikacji, a profilery zapisują do `bench\profiles\` (punkt odniesienia).
- **Nie nadpisuj `bench\results\profile_2026-10-09*` ani `bench\profiles\`**: to punkt odniesienia dla dalszej optymalizacji (porównanie przed/po). Ponowne uruchomienie z tą samą nazwą tylko dopisuje brakujące przebiegi; nowe pomiary zawsze pod nową nazwą (`--name`) albo w innym folderze (`--out`).
- Profilery: GCC: `bash bench/profile_gcc.sh` w WSL (gprof, callgrind → `bench\profiles\`), MSVC: `python bench\profile_msvc.py` (profiler Visual Studio z linii poleceń). Opis wyników: `docs\profilowanie.md`.
- Pomiary: Release, zamknięte inne programy, zasilanie sieciowe; < 5 ms traktuj jako szum.

7.1 Porównanie programów: test A/B i zgodność bitowa

```
python bench\ab.py --list                                                    # stały zestaw 13
przypadków (CA lawa, classic, MC)
python bench\ab.py --bin przed=bench\baseline\engine_przed_msvc.exe --bin
nowy=build\Release\engine.exe --groups ca_lava --out results\ab_test
python bench\ab.py --bin full=build\Release\engine.exe --bin front=build\Release\engine.exe -
-args "full=--set ca.sweep=full" --groups ca_lava --out results\ab_test
python bench\ab.py --wsl --bin przed=bench/baseline/engine_przed_gcc --bin nowy=build-gcc-
release/bin/engine --groups mc --out results/ab_test
python bench\ab.py --from-raw
bench\results\ab\ab_k7_msvc_ca_raw.csv,bench\results\ab\ab_k7_msvc_mc_raw.csv --labels k5,k7
--name ab_razem --out results\ab_test
python bench\compare_results.py --bin przed=bench\baseline\engine_przed_msvc.exe --bin
msvc=build\Release\engine.exe --bin gcc=wsl:build-gcc-release/bin/engine --name zgodnosc_test
--out results\ab_test
python bench\run_profile.py --full --gcc --groups classic,mc,lava,big,detail --cases
test_3d,etna_real --name profil_test --out results\profil_test
python bench\plot_optymalizacja.py                                                    #
docs\img\optymalizacja_*.png z profili _przed2/_po i tabel A/B
```

- `ab.py` uruchamia programy **naprzemiennie** (A B, potem B A), przypięte do procesora 2 (rdzeń P), 5–11 powtórzeń, i po każdym powtórzeniu porównuje wyniki **bitowo** z pierwszym programem; tabela `<nazwa>.md`, mediany `.csv`, każdy przebieg `_raw.csv` w folderze `--out` (domyślnie `bench\results\ab\`; tam są wyniki optymalizacji sekwencyjnej, więc próby kieruj gdzie indziej, np. do `results\`). `--groups ca_lava,ca_classic,mc`, `--cases` (id z `--list`), `--reps`, `--args "etykieta=…"` (np. drugi tryb tego samego programu), `--wsl` (programy GCC w WSL), `--from-raw` (jedna tabela z pomiaru podzielonego na części).
- `compare_results.py` liczy wszystkie `configs\*.json` każdym programem i porównuje foldery wyników bajt po bajcie (przedrostek `wsl:`: program Linux w WSL, więc tabela sprawdza też MSVC ↔ GCC); kod wyjścia 1 przy różnicy.
- Programy odniesienia (`bench\baseline\`) są lokalne (w `.gitignore`); zbuduj je z wybranego commitu i skopiuj tam.
- Presety do porównania flag (optymalizacja sekwencyjna, bez zysku; domyślnych nie zmieniają): `cmake --preset msvc-release-avx2`
- `cmake --build --preset msvc-release-avx2` (program `build-msvc-avx2\Release\engine.exe`) i w WSL `cmake --preset gcc-release-native && cmake --build --preset gcc-release-native -j 4` (`build-gcc-native/bin/engine`, nieprzenośny: instrukcje procesora, na którym kompilujesz).
- `run_profile.py --cases`: tylko wybrane warianty planu (lista: `--dry-run` z tymi samymi `--groups`); profil gorących miejsc po optymalizacji sekwencyjnej: `wsl -e bash -lc "PO_OPTYMALIZACJI=1 bash bench/profile_gcc.sh"` (po zbudowaniu presetu `gcc-profile-noinline`).
- Opis metodyki i wyników: `docs\optymalizacja-sekwencyjna.md`.

7.2 Skalowanie OpenMP

```
python bench\scaling.py --list                                          # przypadki,
harmonogramy, liczby powtórzeń
python bench\scaling.py --cases lava_etna_real --threads 1,2 --reps 1 --min-reps 1 --name
skalowanie_test --out results\skalowanie_test
python bench\scaling.py --cases lava_duza_3d_front --name skalowanie_lawa --out
results\skalowanie_test
python bench\scaling.py --schedules --cases lava_etna_real,lava_duza_3d_front --name
harmonogramy_test --out results\skalowanie_test
python bench\scaling.py --wsl --cases lava_etna_real --threads 1,2 --reps 1 --min-reps 1 --
name skalowanie_gcc_test --out results\skalowanie_test
python bench\scaling.py --from-raw
results\skalowanie_test\skalowanie_test_raw.csv,results\skalowanie_test\skalowanie_lawa_raw.c
sv --name skalowanie_razem --out results\skalowanie_test
python bench\plot_openmp.py                                             #
docs\img\openmp_*.png z bench\results\omp\
python bench\mc_validation.py --from-csv bench\results\omp\mc_walidacja.csv   # tabela i
wykres walidacji szachownicy bez pomiaru
.\build\Release\omp_overhead.exe --pin                                  # mikrotest: region,
bariera, krok z harmonogramami [µs]
```

- `scaling.py` liczy przypadki dla p ∈ {1, 2, 4, 6, 8} wątków (`--threads`) **naprzemiennie**, przypięte do rdzeni P (`parallel.affinity = pcores`), z wysokim priorytetem, ≥ 5 powtórzeń (`--reps`, `--min-reps`). Po każdym powtórzeniu porównuje wyniki wszystkich p bitowo z p = 1. Wynik: `<nazwa>.md` (mediana [min–max], S(p), E(p), część szeregowa f z prawa Amdahla), `.csv`, `_raw.csv`, `_threads.json` (czas pracy wątków z `--thread-stats`; `--no-stats` pomija). Domyślny folder to `bench\results\omp\`; tam są wyniki wersji OpenMP, więc próby kieruj do `results\`.
- `--schedules`: te same przypadki przy 8 wątkach (`--sched-threads`) i 6 harmonogramach (`static`, `static,1`, `dynamic`, `dynamic,4`, `guided`, `guided,4`). `--wsl`: program GCC w WSL (domyślnie `build-gcc-release/bin/engine`; wątki wiąże libgomp). `--from-raw` scala pomiar podzielony na części.
- **Do pomiarów:** zamknięte inne programy (przy 8 wątkach na 8 rdzeniach P każdy program w tle opóźnia wszystkie wątki), zasilanie sieciowe. Pełny zestaw trwa ~30 min; dziel go na części ≤ 10 min (`--cases`) i scalaj `--from-raw`.
- `mc_validation.py` bez `--from-csv` liczy walidację od nowa (12 ziaren generatora, ~1 min) i **nadpisuje** `bench\results\omp\mc_walidacja.*` oraz `docs\img\openmp_mc_walidacja.png`.
- `omp_overhead` (GCC: `build-gcc-release/bin/omp_overhead`): koszt regionu, bariery i kroku „3 fazy po 64 warstwy" dla każdego harmonogramu; `--pin` przypina wątki jak `pcores`, `--reps N`.
- Opis metodyki i wyników: `docs\openmp.md`.

7.3 Skalowanie MPI

```
python bench\scaling_mpi.py --list                                      # przypadki (te same
co w pomiarach OpenMP, bez MC random)
python bench\scaling_mpi.py --cases lava_etna_real --variants m1,m3 --reps 1 --min-reps 1 --
name skalowanie_mpi_test --out results\skalowanie_mpi_test
python bench\scaling_mpi.py --wsl --cases lava_etna_real --variants m1,m3 --reps 1 --min-reps
1 --name skalowanie_mpi_gcc_test --out results\skalowanie_mpi_test
python bench\scaling_mpi.py --from-raw
results\skalowanie_mpi_test\skalowanie_mpi_test_raw.csv --name skalowanie_mpi_razem --out
results\skalowanie_mpi_test
python bench\compare_results.py --set mc.update=checkerboard --bin e=build\Release\engine.exe
--bin m3=build\Release\engine_mpi.exe --launch "m3=mpiexec -n 3" --configs test_3d,lava_mc_2d
--name zgodnosc_mpi_test --out results\zgodnosc_mpi_test
python bench\plot_mpi.py                                                # docs\img\mpi_*.png z
bench\results\mpi\ (i OpenMP z bench\results\omp\)
```

- Warianty (`--variants`, pierwszy = odniesienie zgodności): `m<p>`: p procesów po 1 wątku, `h<p>x<t>`: p procesów po t wątków (tryb hybrydowy), `c<p>`: p procesów z `mpi.exchange = changed`, `o<t>`: `engine` z t wątkami OpenMP (porównanie przy tej samej liczbie rdzeni). Każdy przebieg z `--high-priority` i `parallel.affinity = pcores`, warianty naprzemiennie, ≥ 5 powtórzeń, zgodność bitowa

każdego przebiegu z pierwszym wariantem; tabela: S(p), E(p), udział komunikacji, nierównomierność pasów, bajty na wymianę, Amdahl.

- Domyślny folder to `bench\results\mpi\`; tam są wyniki wersji MPI (`skalowanie_mpi_msvc*`, `skalowanie_mpi_gcc*`), więc próby kieruj do `results\`. Plan pełnego pomiaru (6 części ≤ 6 min, ~30 min razem): `bench\results\mpi\PLAN_POMIARU.md`. Do pomiaru zamknięte inne programy: przy 8 procesach na 8 rdzeniach P każdy program w tle opóźnia cały krok (pasy czekają na najwolniejszy).
- `plot_mpi.py` rysuje schemat podziału, S(p) / E(p), Amdahla, MPI vs OpenMP, tryb hybrydowy, udział komunikacji, czas obliczeń procesów i `changed` vs `always` (`--msvc`, `--gcc`: nazwy pomiarów, domyślnie `skalowanie_mpi_msvc`, `skalowanie_mpi_gcc`). Opis wyników: `docs\mpi.md`, punkt 9.
- `compare_results.py --launch "<etykieta>=<polecenie>"` uruchamia program przez `mpiexec`; `wsl:` przed ścieżką: build GCC w WSL (np. `--bin m3g=wsl:build-gcc-release/bin/engine_mpi --launch "m3g=mpiexec --allow-run-as-root --oversubscribe --bind-to none -n 3"`); programy WSL piszą wyniki na ext4 (`/tmp/volcano_cmp`), do porównania trafia kopia (MPI-IO).

7.4 Typy MPI i MPI-IO

```
python bench\mpi_io.py --list                                                    # 14 przypadków w
grupach pack / bcast / io / read
python bench\mpi_io.py --cases io_bin_classic_2000,io_txt_classic_2000 --variants
m1,m1i,m4,m4i --reps 1 --min-reps 1 --name mpi_io_test --out results\mpi_io_test
python bench\mpi_io.py --wsl --cases io_txt_classic_2000 --variants m4,m4i --reps 1 --min-
reps 1 --name mpi_io_gcc_test --out results\mpi_io_test
python bench\mpi_io.py --wsl --mpiexec "mpiexec --mca io romio321" --cases
io_txt_classic_2000 --variants m4,m4i --reps 1 --min-reps 1 --name mpi_io_romio_test --out
results\mpi_io_test
python bench\mpi_io.py --from-raw results\mpi_io_test\mpi_io_test_raw.csv --name mpi_io_razem
--out results\mpi_io_test
python bench\scaling_mpi.py --cases classic_2000x2000_s400_vn --variants m1,m4,m4d,m4i --reps
1 --min-reps 1 --name skalowanie_dsi_test --out results\mpi_io_test
```

- Warianty jak w `scaling_mpi.py` (`m<p>`: p procesów) z przyrostkami: `d`: `mpi.pack = datatype`, `s`: `mpi.bcast = struct`, `i`: `mpi.io = mpiio` (dowolne razem, np. `m8dsi`, `h2x4d`). Grupy: `pack`: czas wymiany i fazy, bajty i komunikaty, typy; `bcast`: `MPI_Bcast` i budowa configu; `io`: `save`, czas pliku, MB/s, udział w `total`, formatowanie i MPI-IO, pamięć procesu 0; `read`: odczyt `--mc-from`. Każdy przebieg porównany bitowo z pierwszym wariantem przypadku.
- `--wsl`: GCC / OpenMPI, programy i wyniki na ext4 (`/tmp/volcano_mpi_io`); `--mpiexec "mpiexec --mca io romio321"`: ROMIO zamiast domyślnego OMPIO w OpenMPI.
- Domyślny folder to `bench\results\mpi_io\` (plan pomiaru `PLAN_POMIARU.md`, 9 części, ~30 min; wyniki `pomiar_msvc.md`, `pomiar_gcc.md`, `pomiar_gcc_romio.md`), więc próby kieruj do `results\`. Wykresy: `python bench\plot_mpi_io.py` (z `pomiar_*_raw.csv` → `docs\img\mpi_io_*.png`). Opis: `docs\mpi-typy-i-zapis.md`.

7.5 Testy wydajności

```
python bench\zestawienie.py                                                  # zestawienie S / E
optymalizacji, OpenMP, MPI i typów MPI z CSV (bez przebiegów)
python bench\scaling.py --weak --list                                        # przypadki
skalowania słabego
python bench\scaling.py --weak --threads 1,2 --cases weak_lava_front --reps 1 --min-reps 1 --
no-stats --name slabe_test --out results\testy_test
python bench\scaling_mpi.py --weak --variants m1d,m2d --cases weak_classic_2d --reps 1 --min-
reps 1 --name slabe_mpi_test --out results\testy_test
python bench\cell_time.py --curves mc_random --sizes 256,1024 --reps 1 --name krzywa_test --
out results\testy_test
python bench\memory.py --cases classic2d_2000 --name pamiec_test --out results\testy_test
cmake --build build --config Release --target bandwidth; build\Release\bandwidth.exe --mode
stream --threads 1,8 --reps 5 --pin --high-priority
wsl -e python3 bench/cachegrind.py --ll L2,L3 --name cachegrind_test     # symulacja pamięci
podręcznej (WSL, preset gcc-release-g)
python bench\plot_testy.py                                                   #
docs\img\testy_*.png
```

- `--weak`: przestrzeń rośnie z p (classic 2000 × 2000·p, MC szachownica 1000 × 1000·p, lawa 128 × 128·p × 64 na p kaflach terenu); E_w = T(1) / T(p), obok E_w znormalizowana na komórkę × krok. `--wsl`: GCC / OpenMPI.
- Domyślny folder `scaling.py` to `bench\results\omp\`, `scaling_mpi.py`: `bench\results\mpi\`, `cell_time.py` i `memory.py`: `bench\results\testy\` (pomiar testów: `PLAN_POMIARU.md`, ~18 min); próby kieruj do `results\`. `cachegrind.py` nie ma `--out` (zawsze `bench/results/testy/` + `bench/profiles/cachegrind/`, pełny zestaw ~10 min); w próbach zmieniaj `--name`, żeby nie nadpisać `cachegrind.md`. Opis: `docs\testy-wydajnosci.md`.

7.6 Dane zakładki „Wydajność"

```
python tools\wydajnosc_gui.py            # bench\results\ (wszystkie pomiary) -> gui\static\data\wydajnosc.json, bez przebiegów
python tools\wydajnosc_gui.py --check    # czy plik jest aktualny (kod 1: nie; ten sam test: pytest gui\tests\test_perf.py)
python gui\tests\palette_check.py --wykresy   # kontrast i daltonizm kolorów wykresów zakładki (tylko tabela)
```

- Skrypt nie uruchamia silnika: czyta CSV i tabele Markdown pomiarów (lista plików w polu `sources` pliku JSON). Po każdym nowym pomiarze w `bench\results\` (albo zmianie wniosków w skrypcie) uruchom go i odśwież GUI (F5).
- Plik JSON jest w repozytorium, więc zakładka działa także bez danych `bench\results\` (np. w wersji minimalnej). Opis: `docs\zakladka-wydajnosc.md`.

7.7 PDF dokumentów

```
python tools\build_docs_pdf.py               # docs\*.md (bez raportów i spisu README.md) -> docs\pdf\<nazwa>.pdf + docs\pdf\index.json
python tools\build_docs_pdf.py openmp mpi    # wybrane dokumenty (nazwy plików bez .md)
python tools\build_report.py                 # oba raporty: docs\raport-1.md, raport-2.md -> docs\raport-1.pdf, raport-2.pdf
python tools\build_report.py 2 --png results\raport2_strony   # jeden raport (1 albo 2), strony także jako PNG
python tools\raport_wykresy.py 2             # rysunki raportu 2 -> docs\img\raport-2\ (dane z bench\results\)
python gui\tests\raport2_shots.py            # zrzut zakładki „Wydajność" do raportu 2 (sam uruchamia GUI)
```

- Raporty mają osobny skrypt: A4, limit 10 stron (kod 1, gdy więcej), kontrola nazw funkcji i ścieżek z tekstu, a w raporcie 2 także zgodności wierszy listingów z podanym zakresem linii pliku (`--check`: tylko kontrola).
- Wymaga Playwright z Chromium (jak zrzuty GUI) i pakietów `markdown`, `pygments`, `pypdf`; po każdej zmianie dokumentu w `docs\` zbuduj jego PDF ponownie. Skrypt sprawdza obrazki i elementy szersze niż strona (kod 1 przy problemie).
- PDF-y są w repozytorium: link **Pobierz PDF** w zakładce „Wydajność" (`GET /api/doc/pdf`) działa bez budowania, także w wersji minimalnej. Tytuły dokumentów dla nazw pobieranych plików: `docs\pdf\index.json`.

8. Szybki przegląd możliwości (demo, ≈ 6–8 min)

Przygotowanie: zbudowany silnik (2.3), `python -m gui` uruchomione, policzone: `etna_3d` (`.\build\Release\engine.exe --config configs\etna_3d.json --out results\etna_3d`, ~15 s), `etna_real` i `wezuwiusz_real` (punkt 4.1, ~20 s + ~5 s), `lava_mc_3d`, `heightmap_kaldera` (punkt 4) i pełna seria „CA vs MC" (6.1, na wolnym komputerze). Otwarte: przeglądarka z GUI, terminal w katalogu projektu, `docs\profilowanie.md` i `docs\prawdziwe-wulkany.md`.

1. **(1 min) Nowa symulacja → 3D.** *Preset* `etna_3d` → **Uruchom** (pokazać utworzony config i folder) → zakładka **Zadania** (postęp CA: krok i płynna lawa). W tym czasie: **Wyniki** → `etna_3d` (policzony wcześniej) → **3D** → *powierzchnia*, *Pole* krater/ziarno: pięć pól lawy na zboczach; obrót myszą.

2. **(1,5 min) Etna: odtworzenie erupcji suwakiem.** *Nowa symulacja* → *Preset* `etna_real`: panel **Kratery na mapie** (prawdziwy teren z AWS Terrain Tiles, kursor nad szczytem: 3276 m, 37,755° N 14,996° E; kliknięcie dodaje/usuwa krater). **Podgląd 3D** → wynik `etna_real` → *powierzchnia* → zaznacz **odtwarzanie erupcji** → ▶ **Odtwórz** (~15 s): lawa z trzech kraterów szczytowych (jasnożółta → czerwień → ciemny bazalt), liczba komórek płynnych i zastygłych w każdym kroku; zatrzymać na kroku ~60, obrócić kamerę; suwakiem do końca. Komentarz: stan w kroku k liczony z `arrivalStep` / `solidStep` wyniku końcowego (bez ponownej symulacji), test = migawki silnika; lawa schodzi na S i N, zatrzymuje się na górnej ścianie Valle del Bove. Opcjonalnie **Wezuwiusz** (`wezuwiusz_real`, 2D przekrój ⊥ y = 71): wypełnienie krateru i przelew, Monte Somma zatrzymuje lawę. **Zastrzeżenie:** model nie jest skalibrowany fizycznie; to symulacja na prawdziwym terenie, nie prognoza.
3. **(1 min) Przekrój 2D. Podgląd 2D** → `lava_mc_3d`, przekrój ⊥ y, warstwa 40: *Pole* krater (strumienie), potem *kryształ (MC)* + *granice ziaren*: duże kryształy w jeziorze lawy w kraterze, drobne na zboczu („chilled margin": mobilność ze stygnięcia); podpowiedź z czasem stygnięcia komórki.
4. **(1,5 min) Seria z kolejką → wykres CA vs MC. Seria** → szablon „Wpływ sąsiedztwa" (8 zadań, ~3 s) → **Uruchom serię** → **Zadania** (grupa serii, postęp, czas do końca; przed uruchomieniem można *Wstrzymać kolejkę* i pokazać ▲▼ / pauzę) → **Porównanie** → seria „CA vs MC" (policzona wcześniej): wykres log–log 2D i 3D, tabela, galeria; komentarz: MC 4–56× wolniejsze, w 2D przewaga CA maleje z rozmiarem (25× → 4×), w 3D ~30× (`docs\kolejka-serie-dane.md`, 4.2).
5. **(1 min) Czasy, profil. Czasy i statystyki** wyniku: czas każdej fazy, pamięć (pomiar vs model). `docs\profilowanie.md`: najcięższe funkcje (`gatherLavaCandidates` 68–83% lawy, energia MC 60–67%), skalowanie czasu, wymagania sprzętowe (40 B/komórkę → $1000^3 \approx 40$ GB → MPI).
6. **(0,5–1 min) Benchmark vector/macierz.** W terminalu `.\build\Release\engine.exe --benchmark --quick` (~10 s, można uruchomić na początku demo): tabela porównawcza, kolumna „zgodne: tak": obie reprezentacje dają bitowo ten sam wynik.
7. **(0,5 min) Dane we/wy.** *Nowa symulacja* → preset `heightmap_kaldera` (mapa z pliku: kaldera z wyłomem) → **3D** wyniku `heightmap_kaldera`: lawa wypływa tylko przez wyłom; przycisk **VTK** → (opcjonalnie) ParaView. Serwerowo: `python tools\make_series.py --template seeds --dry-run` i `engine --batch`.
8. **(0,5 min) Repozytorium.** Struktura, `README.md`, `docs\README.md` (spis dokumentów), testy (`ctest`, `pytest`).

9. Rozwiązywanie problemów (FAQ)

Problem	Co zrobić
`cmake` nie jest rozpoznawane	CMake nie jest w `PATH`: pełna ścieżka `& "C:\Program Files\CMake\bin\cmake.exe" …` (2.3) albo reinstalacja CMake z „Add CMake to the system PATH"; nowy terminal
`cmake --preset msvc-release`: „could not find any instance of Visual Studio"	brak pakietu „Programowanie aplikacji klasycznych w języku C++": Visual Studio Installer → *Modyfikuj* → zaznacz pakiet
Visual Studio: błędy CMake przy otwieraniu folderu, konfiguracja „x64-Debug" zamiast presetów, „CMake cache" z innym generatorem	wybierz z listy konfiguracji preset `msvc-release`; jeśli dalej błąd, *Projekt → Usuń pamięć podręczną i skonfiguruj ponownie* albo usuń folder `build\` (i `out\`), potem ponownie otwórz folder. Nie mieszaj budowania z linii poleceń i z VS w tym samym `build\` z różnymi generatorami
VS Code nie pyta o preset / buduje Debug	Ctrl+Shift+P → „CMake: Select Configure Preset" → `msvc-release`; w ustawieniach CMake Tools włącz „Use CMake Presets"
`python` otwiera Microsoft Store / nie działa	Python nie jest w `PATH`: reinstalacja z „Add python.exe to PATH" albo *Ustawienia → Aplikacje → Aliasy wykonywania aplikacji* → wyłącz „python.exe"
`pip install`: „Permission denied" / „Defaulting to user installation"	Python dla wszystkich użytkowników: instaluj `python -m pip install --user -r gui\requirements.txt`
`No module named fastapi` / `uvicorn` / `PIL`	`python -m pip install --user -r gui\requirements.txt` (albo dwuklik `gui\run_gui.bat`)
GUI: „Silnik nie znaleziony" (czerwona kropka)	zbuduj Release (2.3); inna ścieżka: `gui\settings.json`: `{"engine": "build/Release/engine.exe"}` albo zmienna `VOLCANO_ENGINE`

Problem	Co zrobić
GUI: ostrzeżenie „Używany jest build Debug”	zbuduj Release; Debug jest kilka razy wolniejszy
`python -m gui`: „Port 8000 jest zajęty”	GUI już działa (otworzy się istniejąca karta) albo inny program zajmuje port: `python -m gui --port 8001`; sprawdzenie: `netstat -ano | findstr :8000`
Strona GUI pusta / błąd modułów JS	stara przeglądarka: użyj aktualnego Chrome, Edge albo Firefox (moduły ES, import maps, WebGL2); twarde odświeżenie Ctrl+F5
Import obrazu mapy: „wymaga pakietu Pillow”	`python -m pip install --user pillow`
WSL: „nie znaleziono ścieżki” / błędy z cudzysłowami	uruchamiaj `wsl -e bash -lc "…"` z katalogu projektu, bez ścieżki i bez zagnieżdżonych cudzysłowów (2.4); jeśli w poleceniu musi być ścieżka Linuksa w Git Bash: `MSYS_NO_PATHCONV=1` przed `wsl`
WSL: `cmake: command not found` / brak `g++`	w Ubuntu: `sudo apt update && sudo apt install -y build-essential cmake valgrind`
WSL wolno buduje / czyta pliki	pliki na `/mnt/d` (dysk Windows) są wolniejsze; do pomiarów `run_profile.py --gcc` kopiuje dane do systemu plików WSL
`engine_mpi` w WSL z `mpi.io=mpiio` zapisuje bardzo wolno	`--out` na dysku Windows (`/mnt/…`, 9p): MPI-IO jest tam 8–16× wolniejsze; pisz do `/tmp/…` (ext4) i skopiuj wynik (punkt 4.2)
Antywirus / SmartScreen blokuje `engine.exe` albo usuwa go po zbudowaniu	dodaj folder `volcano-ca-mc\build` do wyjątków Microsoft Defender (*Zabezpieczenia Windows → Ochrona przed wirusami → Wykluczenia*) i zbuduj ponownie
`engine --batch` kończy się kodem 5	któryś config ma błąd; szczegóły w tabeli w konsoli, w `<out>\batch_summary.csv` (kolumna `error`) i `<out>\<config>\engine.log`
Czasy dużo dłuższe niż w raportach	build Debug, procesy w tle, zasilanie z baterii, rdzeń E; pomaga Release, 1 worker, przypięcie do rdzenia P, zamknięte programy. Stały `WmiPrvSE.exe` w Menedżerze zadań (~2 procesory) to zwykle zapytania WMI programu działającego w tle (np. narzędzia producenta laptopa); zamknij go, najbardziej zwalnia to MC
`pytest` pomija testy („skipped”)	brak zbudowanego silnika: zbuduj Release
`fetch_dem.py`: „nie udało się pobrać kafelka”	brak internetu albo blokada (proxy, zapora); mapy Etny i Wezuwiusza są już w `configs\heightmaps\`; jeśli kafelki były pobrane wcześniej: `--from-cache`
Odtwarzanie erupcji: pole wyszarzone	wynik bez `ca_result.bin` w wersji 2 (np. tylko `ca_result.txt`): uruchom z `output.format = bin` albo `both`
Nagraj WebM nic nie zapisuje	przeglądarka bez `MediaRecorder` dla canvasu (stary Firefox/Safari): użyj Chrome/Edge albo *Klatki PNG (ZIP)*
`screenshots.py` / `run_ca_vs_mc.py`: brak przeglądarki	`python -m pip install --user -r gui\requirements-dev.txt` i `python -m playwright install chromium`

10. Dokumentacja referencyjna: opcje silnika, konfiguracja, pliki wynikowe

10.1 Opcje silnika (`engine`)

Opcja	Znaczenie
`--config <plik>`	konfiguracja JSON; brakujące pola mają wartości domyślne (bez `--config` same domyślne)
`--out <folder>`	folder wyników, tworzony automatycznie (domyślnie `results/<nazwa configu>`)
`--set klucz=wartość`	nadpisanie dowolnego parametru ścieżką z kropkami, można wielokrotnie
`--grid vector|matrix`	reprezentacja siatki (skrót `--set grid=...`)
`--mc-from <folder>`	samo MC na wyniku CA z folderu (`ca_result.bin` v1/v2 lub `ca_result.txt`); parametry MC z configu (`mc.domain = basalt`), wymiary z pliku
`--batch <folder>`	seria: wszystkie configi `*.json` z folderu (kolejność nazw, bez `series.json` i `_*.json`), każdy w osobnym procesie do `<out>/<nazwa>/`, `<out>/batch_summary.csv`; łączy się z `--out`, `--set`, `--grid`, `--timing-detail`, `--progress`
`--benchmark [--repeats N] [--quick]`	porównanie vector vs macierz (patrz niżej)
`--timing-detail`	dodatkowo czasy faz wewnątrz CA (ramka, D(n), reguły, zamiana buforów) i MC (rozrzut czasu MCS, składniki statystyk): sekcja `detail` w `times.json`; domyślnie wyłączone, żeby nie zaburzać pomiaru
`--progress`	(dla GUI) co ~0,5 s linia na stdout: `PROGRESS ca <krok>/<maxSteps> active=<komórki lawy>`, `PROGRESS mc <MCS>/<mcs>`, `PROGRESS save 0/1`; narzut ≤ ~1%, wynik bez zmian

Opcja	Znaczenie
`--high-priority`	proces z wysokim priorytetem (skrót `--set parallel.priority=high`; Windows `HIGH_PRIORITY_CLASS`, Linux nice −5), do pomiarów wieloprocesowych, bo `mpiexec` uruchamia procesy z priorytetem normalnym
`--validate`	(dla GUI) tylko walidacja configu (`--config`, `--set`, `--mc-from`), bez symulacji: JSON `{"ok", "errors", "warnings"}` na stdout, kod 0 lub 1
`--print-defaults`	(dla GUI) pełny domyślny config JSON: wszystkie dozwolone klucze

Symulacja przebiega bez ingerencji użytkownika. Kody wyjścia: 0 OK, 1 błąd konfiguracji, 2 błąd argumentów, 3 inny błąd, 4 niezgodne wyniki w benchmarku, 5 seria (`--batch`) z co najmniej jednym błędnym przebiegiem.

10.2 Konfiguracja (JSON)

```
{
  "space":        { "nx": 200, "ny": 1, "nz": 100 },
  "grid":         "vector",
  "boundary":     "absorbing",
  "neighborhood": "moore",
  "rngSeed":      42,
  "terrain": { "type": "cone", "peakHeight": 70, "baseRadius": 90,
               "craterRadius": 6, "craterDepth": 4, "noise": 0.05, "file": null },
  "vents":   { "count": 1, "placement": "crater", "positions": [], "eruptionSteps": 300,
               "eruptTemp": 1200 },
  "ca":      { "mode": "lava", "maxSteps": 5000, "coolingAmbient": 0.3,
               "coolingLateral": 1.5, "coolingFall": 0.1, "solidusTemp": 800, "ponding": true
},
  "mc":      { "enabled": false, "domain": "basalt", "Q": 50, "mcs": 100, "J": 1.0, "kT":
0.0,
               "mobilityFromCooling": true, "mobilityMin": 0.05, "statsEvery": 10,
"snapshotsEvery": 0 },
  "output":  { "format": "txt", "snapshotsEvery": 0 }
}
```

To są wartości domyślne. Nieznany klucz jest błędem (łapie literówki); klucze zaczynające się od `_` są pomijane (komentarze, np. `"_opis"`), dozwolone są też komentarze `//`.

Parametr	Znaczenie
`space.nx, ny, nz`	wymiary; oś `z` pionowa (z = 0 to spód). 2D = dokładnie jeden wymiar równy 1; w trybie `lava` 2D to przekrój x–z (`ny = 1`)
`grid`	`vector` (płaska tablica) lub `matrix` (`vector<vector<vector>>`)
`boundary`	`absorbing` (ramka = pustka) lub `periodic` (zawijanie; w trybie `lava` tylko osie poziome)
`neighborhood`	`von_neumann` (4/6), `moore` (8/26), `pentagonal` (losowe 5/17), `hexagonal` (losowe 6, tylko 2D)
`rngSeed`	ziarno losowości (teren, kratery, remisy, sąsiedztwa losowe)
`terrain.type`	`cone` (stożek + krater + szum) lub `file` (mapa wysokości CSV w `terrain.file`: `ny` wierszy × `nx` wartości)
`terrain.*`	wysokość szczytu, promień podstawy, promień i głębokość krateru [komórki], amplituda szumu (ułamek wysokości)
`vents.count`	liczba kraterów (lava) / ziaren (classic)
`vents.placement`	`crater` (w kraterze szczytowym), `random` (szczeliny na zboczu) lub `positions` (kolumny z listy)
`vents.positions`	lista kolumn `[[x, y], …]` dla `placement = positions` (krater = pierwsza komórka powietrza nad terenem; `count` = długość listy)
`vents.eruptionSteps, eruptTemp`	jak długo krater ma stałą temperaturę erupcji i jaką [°C]
`ca.mode`	`lava` lub `classic`
`ca.maxSteps`	limit kroków (≤ 65535)
`ca.cooling*`	stygnięcie na krok (`Ambient`), przy rozlewaniu (`Lateral`) i spływie w dół (`Fall`) [°C]
`ca.solidusTemp`	poniżej tej temperatury lawa zastyga w bazalt
`ca.ponding`	wypełnianie zagłębień (krater): rozszerzenie reguł, opis w `engine/src/ca/CaRules.hpp`
`ca.sweep`	`front` (domyślnie): automat frontalny, w kroku tylko komórki, które mogą się zmienić; `full`: pełny przebieg wszystkich komórek (wersja referencyjna); wynik bitowo ten sam
`mc.enabled`	krystalizacja Monte Carlo po CA (albo zamiast CA: `mc.domain = full`)
`mc.domain`	`basalt`: komórki BASALT/LAVA z wyniku CA (wymaga `ca.mode = lava`); `full`: cała przestrzeń, bez terenu i CA

Parametr	Znaczenie
`mc.Q`, `mc.mcs`	liczba orientacji (≥ 2) i kroków Monte Carlo (1 MCS = tyle prób, ile komórek domeny)
`mc.J`, `mc.kT`	energia granicy ziarna (> 0); temperatura Metropolisa (0 = akceptacja tylko ΔE ≤ 0, jak w wersji naiwnej)
`mc.mobilityFromCooling`, `mc.mobilityMin`	mobilność z czasu w stanie płynnym (tylko `basalt`), m ∈ [mobilityMin, 1]
`mc.statsEvery`	co ile MCS liczyć statystyki (`mc_stats.csv`; zawsze też MCS 0 i ostatni)
`mc.snapshotsEvery`	migawki kryształów `mc_snapshots/mcs_<k>.bin` co N MCS (0 = wyłączone; czas poza `mcSim`); odtwarzanie wzrostu ziaren w GUI
`mc.update`	`random` (domyślnie): wariant naiwny, N prób w losowych komórkach (sekwencyjny); `checkerboard`: szachownica, kolory bez sąsiadów tego samego koloru, w MCS każda komórka raz, liczona w wątkach (wynik ten sam dla każdej liczby wątków, statystycznie równoważny `random`)
`parallel.threads`	liczba wątków OpenMP (domyślnie 1: kod sekwencyjny; 0 = `omp_get_max_threads()`); `--threads N` nadpisuje
`parallel.schedule`, `parallel.chunk`	harmonogram pętli po warstwach: `static` / `dynamic` / `guided`; porcja (0 = domyślna OpenMP). Najlepsze wg pomiarów: front lawy `static` z porcją 1, pełny przebieg `dynamic` / `guided`, classic 2D `static`
`parallel.affinity`	`none` (domyślnie: o rdzeniach decyduje system) / `pcores` (jeden wątek na rdzeń fizyczny wydajny P; do pomiarów; w `engine_mpi` proces r z T wątkami na rdzeniach P nr r·T … r·T + T − 1)
`parallel.priority`	`normal` (domyślnie) / `high`: proces sam podnosi priorytet (`--high-priority`)
`mpi.exchange`	(`engine_mpi`) `always` (domyślnie): warstwy brzegowe pasów wymieniane co krok CA / kolor MC; `changed`: warstwa wysyłana tylko, gdy się zmieniła (wynik ten sam, mniej bajtów); `engine` to pole pomija
`mpi.pack`	(`engine_mpi`) `manual` (domyślnie): pola warstw brzegowych kopiowane do bufora i z bufora; `datatype`: typ pochodny MPI opisuje pola w siatce (`MPI_Type_create_struct`, bez bufora); wynik ten sam
`mpi.bcast`	(`engine_mpi`) `json` (domyślnie): proces 0 rozsyła scalony config jako tekst; `struct`: blok liczbowy configu i lista kraterów / zarodków jako struktury MPI; wynik ten sam
`mpi.io`	(`engine_mpi`) `gather` (domyślnie): pasy zbierane na proces 0, który zapisuje pliki (i czyta `--mc-from`); `mpiio`: każdy proces zapisuje / czyta swój pas we wspólnym pliku (MPI-IO; w WSL tylko na ext4); pliki bajtowo te same
`output.format`	`txt`, `bin`, `both` lub `none`; `snapshotsEvery`: migawki `snapshots/step_<k>.bin` (stan 0, co N kroków, ostatni; czas poza `caSim`)

Gotowe konfiguracje (`configs/`):

Plik	Opis
`test_2d.json`	przekrój 200×1×100, 1 krater szczytowy; lawa wypełnia krater i spływa po zboczu
`test_3d.json`	blok 64×64×48, 3 kratery boczne; trzy pola lawy na zboczach
`etna_3d.json`	blok 150×150×80, 5 kraterów bocznych (~15 s)
`classic_ref.json`	klasyczny rozrost 100×100×100, 10 ziaren, von Neumann, periodyczne
`lava_mc_2d.json`	przekrój 240×1×80, szeroki krater; jezioro lawy + jęzory; CA + MC z mobilnością (300 MCS)
`lava_mc_3d.json`	blok 80×80×50, krater szczytowy z jeziorem lawy i przelewem na zbocze; CA + MC (200 MCS)
`mc_full_2d.json`	klasyczny naiwny MC 200×200, Q = 50, 100 MCS, von Neumann, periodyczne
`mc_full_3d.json`	klasyczny naiwny MC 50×50×50, Q = 50, 50 MCS, Moore, periodyczne
`mc_basalt.json`	same parametry MC do `--mc-from`
`heightmap_kaldera.json`	teren z pliku `configs/heightmaps/kaldera_wylom.csv` (100×100×45): kaldera z wyłomem; lawa wypływa tylko przez wyłom; CA + MC (~3 s)
`heightmap_kanion.json`	teren z pliku `kanion.csv`: lawa płynie krętym kanionem
`heightmap_podwojny.json`	teren z pliku `podwojny_wulkan.csv`: lawa spływa na przełęcz i rozdziela się na starszym stożku
`etna_real.json`	**prawdziwa Etna** 160×160×70 (~234 m/komórkę, przewyższenie ×4,2), 3 kratery szczytowe (Voragine, NE, SE) (~20 s)
`etna_real_boczna.json`	Etna, erupcja boczna ze szczeliny na S zboczu (2170–2480 m), inspirowana 2001 r. (~20 s)
`wezuwiusz_real.json`	**prawdziwy Wezuwiusz** z Monte Somma 140×140×50 (~79 m/komórkę), krater szczytowy (~5 s)

10.3 Pliki wynikowe (folder `--out`)

Plik	Zawartość
`config_used.json`	pełna konfiguracja po scaleniu wartości domyślnych i `--set`

Plik	Zawartość
`ca_result.txt`	linia 1: `nx ny nz`, linia 2: `x y z type flowId temp`, potem wszystkie komórki (x najszybciej)
`ca_result.bin`	`"VOLC"`, `uint32 wersja=2`, `int32 nx, ny, nz`, potem tablice `uint8 type[N]`, `int32 flowId[N]`, `float temp[N]`, `uint16 solidStep[N]`, `uint16 arrivalStep[N]` (little-endian, indeks `x + nx*(y + ny*z)`; wersja 1: bez `arrivalStep`, nadal czytana)
`mc_result.txt`	linia 1: `nx ny nz`, linia 2: `x y z crystal`, potem wszystkie komórki (`crystal` = orientacja 1..Q, 0 poza domeną MC)
`mc_result.bin`	`"VMC1"`, `int32 nx, ny, nz`, `int32 crystal[N]` (przy `output.format` = `bin` / `both`)
`mc_stats.csv`	`mcs,grains,meanGrainSize,energy,acceptRate` co `mc.statsEvery` MCS
`times.json`	czasy [ms]: `readConfig, allocate, terrain, vents, caSim, caStats, save` (+ `mcInit, mcSim, mcStats, mcSave`; przy `--mc-from` `readInput`), `total` + `steps, cells, grid, neighborhood, boundary, mode, cpu, compiler, build` (+ `mcs, domainCells, Q, mcDomain, mcMobility`). OpenMP: `threads, schedule, chunk, affinity, cpus` (procesory logiczne kolejnych wątków przy `affinity = pcores`), `openmp` (wersja; 0 = build bez OpenMP); z `--thread-stats`: `threadStats` (czas pracy każdego wątku w fazach kroku, `imbalance` = Σ maks / Σ średnia). W trybie `mc.domain = full` sekcji CA nie ma (`mode = mc_full`). `engine_mpi`: czasy procesu 0 i sekcja `mpi`: procesy, wątki na proces, biblioteka, `exchange`, oś i podział warstw (`split`), `initMs` (start MPI, wliczony w `total`), `bcast` (bajty i czas rozsyłania configu i terenu), `ca` / `mc` (czas fazy, komunikacji, wymiany warstw, `MPI_Allreduce` i obliczeń: min / średnia / maks i `perRank`; wymiany, komunikaty, bajty, warstwy pominięte; MC: statystyki ziaren, zebranie mapy, etykiety na procesie 0), `gather`, `cpus`, `priority`, `peakRssBytes`; typy MPI i MPI-IO:
`pack`, `bcast.mode` (+ `configPrepMs`, `configBuildMs`, `ventsBytes`, `ventsMs`), `ca` / `mc`: `typesBuilt`,	
`typeBuildMs`, `ca.gatherMs` (pas bez ramki + zebranie), `io` (`mode`; `files`: każdy plik zapisany przez MPI-IO:	
bajty, `formatMs` i `ioMs` min / średnia / maks po procesach, `ms`, `MBps`; `read`: odczyt `--mc-from`), `scatter`;	
`save` liczy się w obu trybach od końca fazy CA. Profil: `files`: rozmiar [B] i czas zapisu [ms] każdego pliku; `memory`: pamięć procesu na starcie, na końcu i szczytowa (`peakRssBytes`: Windows `PeakWorkingSetSize`, Linux `ru_maxrss`; `peakPrivateBytes`: Windows `PeakPagefileUsage`) oraz `model`: teoretyczne zapotrzebowanie z tablic × `sizeof` (szczyt, bajty na komórkę, skład szczytu); `detail`: tylko z `--timing-detail`	
`stats.json`	objętość lawy na krater, zasięg (odległość pozioma od krateru), liczba komórek LAVA i zmienionych w każdym kroku; sekcja `mc`: ziarna i średnia wielkość na początku/końcu, energia, efekt mobilności (`byMobility`)

Typy komórek: 0 powietrze, 1 skała, 2 lawa (w `classic`: ziarno), 3 bazalt.

10.4 Benchmark: vector vs macierz

`engine --benchmark` uruchamia te same symulacje na obu reprezentacjach siatki (naprzemiennie, `--repeats` razy, mediana czasu samego automatu), porównuje wyniki komórka po komórce (bitowo) i wypisuje tabelę porównawczą (+ kolumna `tryb` = tryb/sąsiedztwo) oraz zapisuje `benchmark.csv` i `benchmark_meta.json` (procesor, kompilator). Przypadki: rozrost klasyczny (von Neumann, periodyczne) $10^3/10$, $25^3/62$, $50^3/100$, $100^3/100$, $100^2/10$, $1000^2/100$ (+ 10^3 i 25^3 z Moore) oraz lawa 200×1×100 i 64×64×48. `--quick` pomija przypadki ≥ 1 mln komórek.

10.5 Podgląd wyniku bez GUI (`tools/plot_slice.py`, `tools/plot_mc_stats.py`)

Podgląd wyniku (PNG):

```
python tools\plot_slice.py results\test_2d                      # przekrój y = 0, kolor wg
typu
python tools\plot_slice.py results\test_3d --axis top --color flow   # widok z góry, kolor wg
krateru
python tools\plot_slice.py results\test_3d --axis y --index 32 --color temp
python tools\plot_slice.py results\mc_full_2d --color crystal --boundaries       # kryształy
MC + granice
python tools\plot_slice.py results\lava_mc_2d --color crystal --window 60 150 20 56
python tools\plot_slice.py results\lava_mc_3d --axis top --color cooling        # czas w
stanie płynnym
python tools\plot_mc_stats.py results\lava_mc_3d results\lava_mc_3d_nomob --labels "z
mobilnością" "bez"
```

`plot_slice.py`: bez `--axis` dla 2D rysuje płaszczyznę siatki; `--color` = `type`, `flow`, `temp`, `cooling` (solidStep – arrivalStep), `crystal` (mapa kryształów MC, stały pseudolosowy kolor ID; `--boundaries`: granice ziaren); `--crop` / `--window H0 H1 V0 V1` przycinają obraz. `plot_mc_stats.py <folder> [...]`: liczba ziaren i średnia wielkość ziarna vs MCS z `mc_stats.csv`, kilka folderów na jednym wykresie (legenda z nazw folderów lub `--labels`).